Tyr-Crypto Computational Handbook

Volume 6: Gimli and generic sponges

A guided calculation book for the 384-bit Gimli permutation, column rounds, swap schedule, 16-byte-rate absorption, padding, squeezing, domain framing, tags, and streams.

Assumed knowledge: Volumes 0 and 3. SHA3 introduced the general absorb-permute-squeeze picture; this volume rebuilds it for Gimli.

Blue	new input, public data, or plain-language explanation
Purple	exact recipe to follow
Green	fully worked calculation
Orange	exercise for the reader
Red	misuse, security, or implementation danger
Teal	checkpoint before continuing

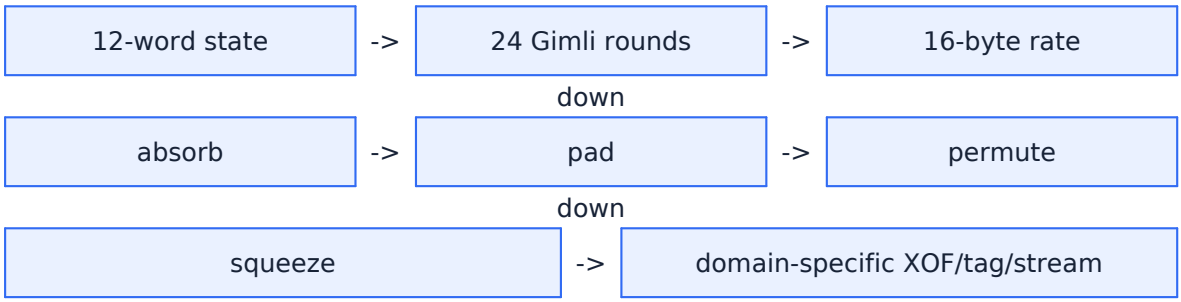
Repository snapshot reviewed: main branch, July 2026.
Square-corner calculation-first teaching edition.

1 What this volume will teach

Read this first

This volume does not ask you to memorize an algorithm diagram. Each page introduces exactly one mechanical action. First read the plain-language box, then copy the rule, then cover the worked examples and reproduce them yourself.

The full path



Study routine for an easily overloaded brain

1. Work for one method only. 2. Say every intermediate value aloud or write it down. 3. Stop at the teal checkpoint. 4. Continue only when the checkpoint feels boring. There is no reward for carrying an unclear step into the next page.

Vocabulary before calculations

Permutation	A reversible fixed-size transformation of the whole state.
Rate	The part of a sponge state directly exposed to input and output; here 16 bytes.
Capacity	The hidden remainder of the state; here 32 bytes.
Column round	The nonlinear transformation applied independently to each of four three-word columns.
Small swap	Swap adjacent top-row pairs and inject a round constant.
Big swap	Exchange the left and right halves of the top row.
Domain separation	Use different fixed prefixes so the same bytes mean different things in XOF, tag, and stream modes.
Framing	Encode field lengths so distinct key/nonce/message tuples remain distinct.

2 Algorithm overview before the details

The promise of the overview

The boxes below deliberately hide most arithmetic. Their job is only to show where each later calculation belongs. When you meet a calculation page, return here and point to its box.

1	12-word state
2	24 Gimli rounds
3	16-byte rate
4	absorb
5	pad
6	permute
7	squeeze
8	domain-specific XOF/tag/stream

Important scope

The examples often use toy-sized states or selected words. The toy calculation keeps the same order of operations but uses smaller numbers so that a human can finish it. Every toy rule is explicitly marked. Never substitute toy parameters into real cryptographic code.

3 Method 1: lay out the 384-bit Gimli state

Input:	twelve 32-bit words
Do:	arrange three rows of four columns
Output:	a 3x4 word state

Plain-language explanation

Gimli stores twelve words in one array. Indices 0..3 are the top row, 4..7 the middle row, and 8..11 the bottom row. Column c therefore contains indices c,4+c,8+c.

Mechanical rule

Top row: s0 s1 s2 s3. Middle: s4 s5 s6 s7. Bottom: s8 s9 s10 s11. Column c=(s[c],s[4+c],s[8+c]).

Worked example 1 - column 0

Indices 0,4,8.

Worked example 2 - column 1

Indices 1,5,9.

Worked example 3 - column 2

Indices 2,6,10.

Worked example 4 - column 3

Indices 3,7,11.

Exercises

(a) List the three indices of column 2.
(b) Which row holds s6?
(c) What is the bottom word of column 1?
(d) Write the row groups for indices 0..11.

Checkpoint

You can locate a word from row and column without drawing all twelve boxes.

4 Method 2: rotate the top and middle words of a column

Input:	top word a and middle word b
Do:	rotate a left 24 and b left 9
Output:	temporary words x and y

Plain-language explanation

Every column transformation begins with two rotations. Rotation wraps bits around; it does not discard them. The bottom word z is copied unchanged for this preparation step.

Mechanical rule

x=ROTL32(a,24). y=ROTL32(b,9). z=c. ROTL24 can be viewed as moving the lowest byte to the highest position while shifting the other bytes down.

Worked example 1 - a=0x11223344

ROTL24 -> 0x44112233.

Worked example 2 - a=0x00000001

ROTL24 -> 0x01000000.

Worked example 3 - b=0x00000001

ROTL9 -> 0x00000200.

Worked example 4 - b=0x80000000

ROTL9 wraps the top bit -> 0x00000100.

Exercises

(a) ROTL24(0xAABBCCDD).
(b) ROTL24(0x00000080).
(c) ROTL9(0x00000002).
(d) Why is rotation different from left shift?

Checkpoint

You explicitly wrap the bits that pass the left edge back to the right edge.

5 Method 3: apply the nonlinear Gimli column transformation

Input:	three words in one column
Do:	prepare x,y,z, then calculate three shifted/XOR expressions
Output:	three replacement words

Plain-language explanation

The new bottom, middle, and top are calculated from the old prepared x,y,z. Do not update the state array after the first formula and then use that new value in the next formula. All three formulas use the same old x,y,z.

Mechanical rule

x=ROTL24(top), y=ROTL9(mid), z=bottom. newBottom=x XOR (z<<1) XOR ((y AND z)<<2). newMiddle=y XOR x XOR ((x OR z)<<1). newTop=z XOR y XOR ((x AND y)<<3). Keep low 32 bits.

Worked example 1 - column 0x00000000,0x00000000,0x00000000

x=ROTL24(top)=0x00000000; y=ROTL9(middle)=0x00000000; z=bottom=0x00000000. New top=0x00000000, middle=0x00000000, bottom=0x00000000.

Worked example 2 - column 0x00000001,0x00000000,0x00000000

x=ROTL24(top)=0x01000000; y=ROTL9(middle)=0x00000000; z=bottom=0x00000000. New top=0x00000000, middle=0x03000000, bottom=0x01000000.

Worked example 3 - column 0x12345678,0x9ABCDEF0,0x0F1E2D3C

x=ROTL24(top)=0x78123456; y=ROTL9(middle)=0x79BDE135; z=bottom=0x0F1E2D3C. New top=0xB622CCA9, middle=0xFF93AF9F, bottom=0x425EEAFE.

Worked example 4 - column 0xFFFFFFFF,0x80000000,0x00000001

x=ROTL24(top)=0xFFFFFFFF; y=ROTL9(middle)=0x00000100; z=bottom=0x00000001. New top=0x00000901, middle=0x00000101, bottom=0xFFFFFFFD.

Exercises

(a) Transform column (0,0,1).
(b) Transform column (1,1,1).
(c) In which formula is OR used?
(d) Why must x,y,z be saved before writing results?

Checkpoint

You can calculate newBottom, newMiddle, and newTop in any order and get the same answer.

6 Method 4: perform the small swap and inject the round constant

Input:	top row after column transformations and a round divisible by 4
Do:	swap adjacent pairs and XOR a public constant into s0
Output:	modified top row

Plain-language explanation

After every column has been transformed, some rounds permute the top row. When round mod 4 equals 0, Gimli performs the small swap and injects 0x9E377900 OR round into the new s0.

Mechanical rule

For round r with r mod 4=0: swap s0<->s1 and s2<->s3. Then s0 = s0 XOR (0x9E377900 OR r).

Worked example 1 - small swap only

[A B C D] -> [B A D C] before constant injection.

Worked example 2 - round 24 constant

0x9E377900 OR 0x18 = 0x9E377918.

Worked example 3 - round 20 constant

0x9E377900 OR 0x14 = 0x9E377914.

Worked example 4 - numeric row

Round 24, row [0,1,2,3] -> swap [1,0,3,2]; s0=1 XOR 9E377918 = 9E377919.

Exercises

(a) Small-swap [10 20 30 40].
(b) Constant for round 16.
(c) Round 8 row [0,0,0,0]: final s0?
(d) Does the constant alter the middle or bottom row?

Checkpoint

You first swap, then inject the constant into the already-swapped s0.

7 Method 5: perform the big swap

Input:	top row and a round congruent to 2 modulo 4
Do:	exchange the left and right halves
Output:	modified top row

Plain-language explanation

When round mod 4 equals 2, there is no constant injection. The big swap moves columns 0 and 1 to the right half and columns 2 and 3 to the left half.

Mechanical rule

[s0,s1,s2,s3] -> [s2,s3,s0,s1] for rounds 22,18,14,10,6,2.

Worked example 1 - letters

[A B C D] -> [C D A B].

Worked example 2 - numbers

[00 11 22 33] -> [22 33 00 11].

Worked example 3 - round 22

22 mod 4=2, so big swap occurs.

Worked example 4 - round 21

21 mod 4=1, so no top-row swap occurs.

Exercises

(a) Big-swap [10 20 30 40].
(b) Does round 18 swap? Which type?
(c) Does round 12 use big or small swap?
(d) List all swap types for rounds 8,7,6,5.

Checkpoint

From r mod 4 alone, you can decide small swap, big swap, or none.

8 Method 6: classify all 24 Gimli rounds

Input:	round number from 24 down to 1
Do:	apply column transforms, then choose swap by round mod 4
Output:	one complete permutation schedule

Plain-language explanation

Every round always transforms all four columns. The modulo-4 result controls only the top-row rearrangement afterward. The implementation counts downward from 24 to 1.

Mechanical rule

Each round: four column transforms. If r mod4=0: small swap + constant. If r mod4=2: big swap. Otherwise no swap. Then decrement r.

Worked example 1 - round 24

Columns -> small swap -> inject 0x9E377918.

Worked example 2 - round 23

Columns -> no swap.

Worked example 3 - round 22

Columns -> big swap.

Worked example 4 - round 21

Columns -> no swap. Then the four-round pattern repeats at 20.

Exercises

(a) Classify round 17.
(b) Classify round 16.
(c) Classify round 14.
(d) How many small-swap rounds occur in 24 rounds?

Checkpoint

You can write the repeating pattern small, none, big, none while counting downward.

9 Method 7: absorb one 16-byte rate block

Input:	16 input bytes and current state
Do:	read four little-endian words, XOR them into s0..s3, then permute
Output:	updated 384-bit state

Plain-language explanation

Tyr-Crypto’s Gimli sponge exposes only the first four words to input and output. Those sixteen bytes are the rate. The remaining eight words are capacity: they are changed by the permutation but not directly XORed with the message.

Mechanical rule

Parse bytes 0..3 as w0 little-endian, 4..7 as w1, 8..11 as w2, 12..15 as w3. Set s[i]=s[i] XOR wi for i=0..3. Then apply Gimli permutation.

Worked example 1 - first word

Bytes 01 02 03 04 -> w0=0x04030201.

Worked example 2 - XOR into zero state

Zero s0 XOR 0x04030201 -> 0x04030201 before permutation.

Worked example 3 - XOR into nonzero

s1=0xFFFFFFFF, w1=0x00000001 -> 0xFFFFFFFE.

Worked example 4 - capacity untouched directly

s4..s11 receive no message XOR during absorption; they change only through permutation.

Exercises

(a) Parse 10 20 30 40 as w0.
(b) s2=AAAAAAAA, w2=55555555: new s2?
(c) How many words are directly XORed per rate block?
(d) How many state bytes are capacity?

Checkpoint

You can separate “absorb by XOR” from “mix everything by permutation.”

10 Method 8: pad the final partial block

Input:	0 to 15 unabsorbed message bytes
Do:	append domain byte 0x1F and toggle 0x80 at rate end
Output:	one complete 16-byte final block

Plain-language explanation

A sponge must encode where the message ends. Tyr-Crypto zero-fills the unused rate bytes, XORs 0x1F at the first unused position, and XORs 0x80 into byte 15. If those positions coincide, both values are XORed into the same byte.

Mechanical rule

Start with a zero 16-byte buffer containing the remaining bytes. buffer[len] XOR=0x1F. buffer[15] XOR=0x80. Absorb and permute.

Worked example 1 - empty tail

len=0 -> byte0=1F and byte15=80.

Worked example 2 - one byte

[AA] -> AA 1F 00...00 80.

Worked example 3 - fifteen bytes

len=15 -> final byte becomes 1F XOR 80 = 9F.

Worked example 4 - four-byte tail

11 22 33 44 -> then 1F at index4, zeros, 80 at index15.

Exercises

(a) Pad empty tail.
(b) Pad bytes 01 02.
(c) What is final byte for a 15-byte tail?
(d) Why is a full 16-byte block permuted before a separate padded block?

Checkpoint

You always place the 0x1F marker at the first unused byte, even for an empty message.

11 Method 9: squeeze output from the rate

Input:	a permuted state in squeeze mode and requested length
Do:	serialize s0..s3 little-endian; permute again whenever 16 bytes are exhausted
Output:	output bytes

Plain-language explanation

Squeezing is the reverse direction of the sponge interface: it reads the first sixteen state bytes. It does not remove them. For more than sixteen output bytes, the state is permuted and a fresh rate block is serialized.

Mechanical rule

Write s0,s1,s2,s3 as four little-endian words. Copy requested bytes. When offset reaches 16 and more output is needed, permute state and refill the 16-byte rate buffer.

Worked example 1 - one word

s0=0x04030201 serializes as 01 02 03 04.

Worked example 2 - 8-byte output

Copy all four bytes of s0 and all four of s1.

Worked example 3 - 16-byte output

Copy exactly one rate block; no second permutation is needed.

Worked example 4 - 20-byte output

Copy 16 bytes, permute, then copy first 4 bytes of the next rate block.

Exercises

(a) How many squeeze blocks for 1 byte?
(b) How many for 17 bytes?
(c) Serialize 0xA0B0C0D0 little-endian.
(d) Does squeezing expose capacity words s4..s11 directly?

Checkpoint

You calculate squeeze-block count as ceil(outLen/16).

12 Method 10: frame key, nonce, and message with lengths

Input:	domain string plus key, nonce, and message
Do:	absorb a domain and an 8-byte little-endian length before each field
Output:	an unambiguous byte sequence

Plain-language explanation

Simply concatenating key, nonce, and message can be ambiguous: different tuples may produce the same concatenation. Tyr-Crypto's keyed Gimli calls prevent this by encoding the length of each field before the field itself.

Mechanical rule

For keyed calls absorb: domain || LE64(keyLen) || key || LE64(nonceLen) || nonce || LE64(messageLen) || message. For unkeyed XOF with empty key and nonce, absorb the message directly.

Worked example 1 - length 1

LE64(1)=01 00 00 00 00 00 00 00.

Worked example 2 - length 24

LE64(24)=18 00 00 00 00 00 00 00.

Worked example 3 - avoid ambiguity

(key=A, nonce=BC) and (key=AB, nonce=C) receive different length fields.

Worked example 4 - domain separation

XOF, TAG, and STRM use different fixed 16-byte domain strings before the framed fields.

Exercises

(a) Encode LE64(32).
(b) Encode LE64(256).
(c) Why are field lengths included?
(d) What shortcut is used for an unkeyed XOF with empty key and nonce?

Checkpoint

You can write the exact absorbed field order without omitting a length.

13 Method 11: compute a Gimli tag

Input:	32-byte key, 24-byte nonce, message, and tag length
Do:	use TAG domain, framed absorb, padding, permutation, and squeeze
Output:	a deterministic tag byte string

Plain-language explanation

The tag helper is a keyed sponge output under a tag-specific domain. The same key and nonce with a different message should produce a different tag. Verification must compare tags in constant time at a higher layer.

Mechanical rule

Require key length 32 and nonce length 24. Absorb TAG domain and framed fields. Pad, permute, then squeeze outLen bytes; default outLen is 16.

Worked example 1 - default tag

outLen omitted -> request 16 bytes.

Worked example 2 - empty message

Message length field is eight zero bytes; the key and nonce are still absorbed.

Worked example 3 - 32-byte tag

Squeeze two 16-byte rate blocks.

Worked example 4 - wrong nonce length

A 23-byte nonce is rejected before sponge processing.

Exercises

(a) How many squeeze blocks for default tag?
(b) What key length is required?
(c) What nonce length is required?
(d) Does tag mode use the same domain as stream mode?

Danger to remember

A tag alone does not encrypt. A stream alone does not authenticate. Combining custom primitives safely requires a defined composition, nonce policy, and verification-before-release rule.

Checkpoint

You can list validation, framing, padding, permutation, and squeeze in order.

14 Method 12: generate and apply the Gimli stream

Input:	32-byte key, 24-byte nonce, and input
Do:	generate a STRM-domain XOF of matching length, then XOR
Output:	ciphertext or recovered plaintext

Plain-language explanation

The stream helper first generates exactly as many keystream bytes as the input length. It uses an empty message field in the domain-separated XOF and then XORs each input byte with the keystream. Applying the same key, nonce, and operation again reverses it.

Mechanical rule

KS = GimliXOFWithDomain(key, nonce, emptyMessage, STRM, inputLen). output[i]=input[i] XOR KS[i].

Worked example 1 - one byte

Input 4D and keystream A6 -> output EB.

Worked example 2 - reverse

EB XOR A6 -> 4D.

Worked example 3 - four bytes

10 20 30 40 XOR 01 02 04 08 -> 11 22 34 48.

Worked example 4 - 20 bytes

Generate 20 XOF bytes: one full 16-byte squeeze block plus 4 bytes after another permutation.

Exercises

(a) Input 00 FF with KS AA 55.
(b) Reverse AA AA with KS 0F F0.
(c) How many squeeze blocks for 33 input bytes?
(d) What message is absorbed in the current stream-XOF call?

Danger to remember

Repeating a key/nonce pair repeats the keystream. The current stream helper is a confidentiality primitive, not an authenticated-encryption API.

Checkpoint

You can distinguish the absorbed framing message (empty) from the bytes later XORed with the generated stream.

15 Cumulative guided calculations

How cumulative work differs

Earlier exercises isolated one action. These tasks chain several actions together. Draw a small checkbox after every line. Do not calculate two lines in your head at once.

Cumulative task 1 - One round classification

For rounds 12,11,10,9, list the column step and top-row action.

Cumulative task 2 - Absorb a 5-byte tail

Tail is 01 02 03 04 05. Write the complete padded 16-byte block before absorption.

Cumulative task 3 - Tag framing sizes

Key=32 bytes, nonce=24 bytes, message=3 bytes. Write the non-secret length encodings in order.

Cumulative task 4 - Stream length

For a 45-byte input, state how many squeeze blocks are generated and how much of the final block is used.

16 Tyr-Crypto code map

How to read the code map

The left column names the calculation from this volume. The right column names the current Nim location or implementation behavior. This is a map, not a claim that the implementation is independently audited.

Permutation column update	src/protocols/custom_crypto/symmetric/gimli/gimli.nim: gimliColumnRound
24-round schedule	gimli.nim: gimli_core_ref
SIMD permutation dispatch	gimli.nim and gimli_sse.nim
Sponge state and 16-byte rate	src/protocols/custom_crypto/symmetric/gimli/gimli_sponge.nim
Absorb and padding	gimli_sponge.nim: gimliAbsorbUpdate, gimliAbsorbFinal
Squeeze	gimli_sponge.nim: gimliSqueezeInto
Length framing and domains	gimli_sponge.nim: absorbFramedInput and TYR-GIMLI-* domain arrays
Tag and stream helpers	gimli_sponge.nim: gimliTag, gimliStreamXor

Security and implementation note

The current repository uses a 16-byte rate and 32-byte capacity, with 0x1F/0x80 byte-aligned padding. Its XOF, tag, and stream interfaces use repository-specific 16-byte domain strings and explicit key/nonce/message length framing.

Security and implementation note

Gimli is a permutation; security properties belong to the complete sponge or mode around it. The repository-specific tag and stream helpers should be treated as experimental custom constructions, particularly when considering composition, nonce reuse, and independent analysis.

17 Compact solutions with paths

Use solutions correctly

First compare the final answer. If it differs, compare one intermediate value at a time from left to right. Do not copy the whole line: locate the first point where your work diverged.

Method 1: lay out the 384-bit Gimli state

Solution path

(a) 2,6,10.
(b) Middle row.
(c) s9.
(d) Top 0..3; middle 4..7; bottom 8..11.

Method 2: rotate the top and middle words of a column

Solution path

(a) 0xDDAABBCC.
(b) 0x80000000.
(c) 0x00000400.
(d) A shift discards outgoing bits and inserts zeros; a rotation returns outgoing bits at the other end.

Method 3: apply the nonlinear Gimli column transformation

Solution path

(a) 0x00000001, 0x00000002, 0x00000002.
(b) 0x00000201, 0x03000202, 0x01000002.
(c) The newMiddle formula: (x OR z)<<1.
(d) Otherwise a later formula might accidentally use a new state word instead of the old prepared value.

Method 4: perform the small swap and inject the round constant

Solution path

(a) 20 10 40 30.
(b) 0x9E377910.
(c) 0x9E377908.
(d) No; only top-row word s0.

Method 5: perform the big swap

Solution path

(a) 30 40 10 20.
(b) Yes, big swap.
(c) Small swap, because 12 mod 4=0.
(d) 8 small; 7 none; 6 big; 5 none.

Method 6: classify all 24 Gimli rounds

Solution path

(a) No swap, because 17 mod4=1.
(b) Small swap + constant.
(c) Big swap.
(d) 6 small-swap rounds: 24,20,16,12,8,4.

Method 7: absorb one 16-byte rate block

Solution path

(a) 0x40302010.
(b) 0xFFFFFFFF.
(c) 4 words.
(d) 32 bytes: eight 32-bit words.

Method 8: pad the final partial block

Solution path

(a) 1F 00...00 80.
(b) 01 02 1F 00...00 80.
(c) 0x9F.
(d) The full block has no unused position for the message-end marker; padding requires another block.

Method 9: squeeze output from the rate

Solution path

(a) 1.
(b) 2.
(c) D0 C0 B0 A0.
(d) No. Only s0..s3 are serialized as rate bytes.

Method 10: frame key, nonce, and message with lengths

Solution path

(a) 20 00 00 00 00 00 00 00.
(b) 00 01 00 00 00 00 00 00.
(c) To prevent distinct tuples from sharing the same concatenated bytes.
(d) It absorbs the message directly without the keyed framing fields.

Method 11: compute a Gimli tag

Solution path

(a) 1 block.
(b) 32 bytes.
(c) 24 bytes.
(d) No. TAG and STRM have distinct domain strings.

Method 12: generate and apply the Gimli stream

Solution path

(a) AA AA.
(b) A5 5A.
(c) 3 blocks.
(d) An empty message; input bytes are not absorbed, they are XORed afterward.

18 Cumulative solutions

Cumulative solution 1 - One round classification

All four perform four column transforms. Round 12: small swap + constant 0x9E37790C. Round 11: none. Round 10: big swap. Round 9: none.

Cumulative solution 2 - Absorb a 5-byte tail

01 02 03 04 05 1F followed by nine 00 bytes, then 80 at index 15.

Cumulative solution 3 - Tag framing sizes

LE64(32)=20 00 00 00 00 00 00 00; LE64(24)=18 00 00 00 00 00 00 00; LE64(3)=03 00 00 00 00 00 00 00.

Cumulative solution 4 - Stream length

Three 16-byte blocks are generated. The first two supply 32 bytes and 13 bytes of the third are used.

19 Primary references

Tyr-Crypto Gimli permutation	https://github.com/siriuslee69/Tyr-Crypto/blob/main/src/protocols/custom_crypto/symmetric/gimli/gimli.nim
Tyr-Crypto Gimli sponge	https://github.com/siriuslee69/Tyr-Crypto/blob/main/src/protocols/custom_crypto/symmetric/gimli/gimli_sponge.nim
Gimli: a cross-platform permutation	https://gimli.cr.yp.to/gimli-20170627.pdf
Gimli project site	https://gimli.cr.yp.to/

Safety boundary

This handbook explains calculations and implementation structure. It does not certify Tyr-Crypto or any custom cryptographic implementation for production use. Protocol composition, randomness, nonce management, compiler behavior, side channels, key erasure, and independent review remain separate requirements.